AM 234 / BME 234:
Machine Learning and Artificial Intelligence in Genomics

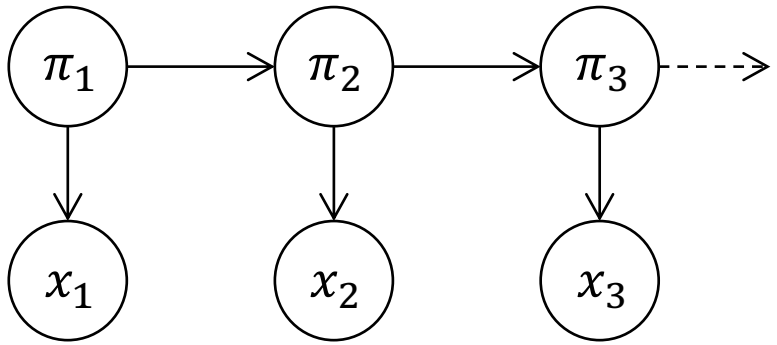
Class 4: April 9, 2026

Spring Quarter 2026

Reminders from last class

Hidden states follow a Markov chain
with transition probabilities:

$$a_{kl} = P(\pi_i = l | \pi_{i-1} = k)$$



Observations (emissions) depend only
on the corresponding hidden state:

$$e_k(b) = P(x_i = b | \pi_i = k)$$

(emission probabilities)

Reminders from last class

Joint probability of an observed sequence and hidden state sequence, given the model parameters:

$$P(x,\pi) = a_{0\pi_1} \prod_{i=1}^{L} e_{\pi_i}(x_i) a_{\pi_i \pi_{i+1}}$$

Probability of starting in hidden state π_1

Product over all emissions and transitions from each hidden state

Reminders from last class

Decoding: Finding the most likely hidden states given the observations and model parameters.

1. Viterbi algorithm:
   Finds the most likely sequence of hidden states.

$$\pi^* = \underset{\pi}{\operatorname{argmax}}\, P(x, \pi)$$

2. Forward-backward algorithm (posterior decoding):
   Finds the most likely hidden state at each position.

$$\hat{\pi}_i = \underset{k}{\operatorname{argmax}}\, P(\pi_i = k|x)$$

How to find parameters for an HMM?

Our previous discussion of finding the most likely hidden states assumed that model parameters were known.

Transition probabilities: $a_{kl} = P(\pi_i = l | \pi_{i-1} = k)$

Emission probabilities: $e_k(b) = P(x_i = b | \pi_i = k)$

Options for learning parameters:

1. Supervised learning: given a set of sequences labeled with hidden states (training data).
2. Unsupervised learning: only unlabeled sequences are available.

How to find parameters for an HMM?

Goals for parameter estimation:

We want parameters that maximize the likelihood of a set of n (independent) observed sequences $x^{(1)}, \ldots, x^{(n)}$:

$$P\left(x^{(1)}, \ldots, x^{(n)} \middle| \theta\right) = \prod_{j=1}^{n} P\left(x^{(j)} \middle| \theta\right)$$

$$\hat{\theta} = \text{argmax}_{\theta} \prod_{j=1}^{n} P\left(x^{(j)} \middle| \theta\right)$$

Parameter estimation with hidden state labels

Assume hidden state sequences are known for some examples (i.e. training sequences are labeled).

In this case we can count all transitions and emissions:

A_{kl} = # of transitions from state k to state l in the labeled training sequences

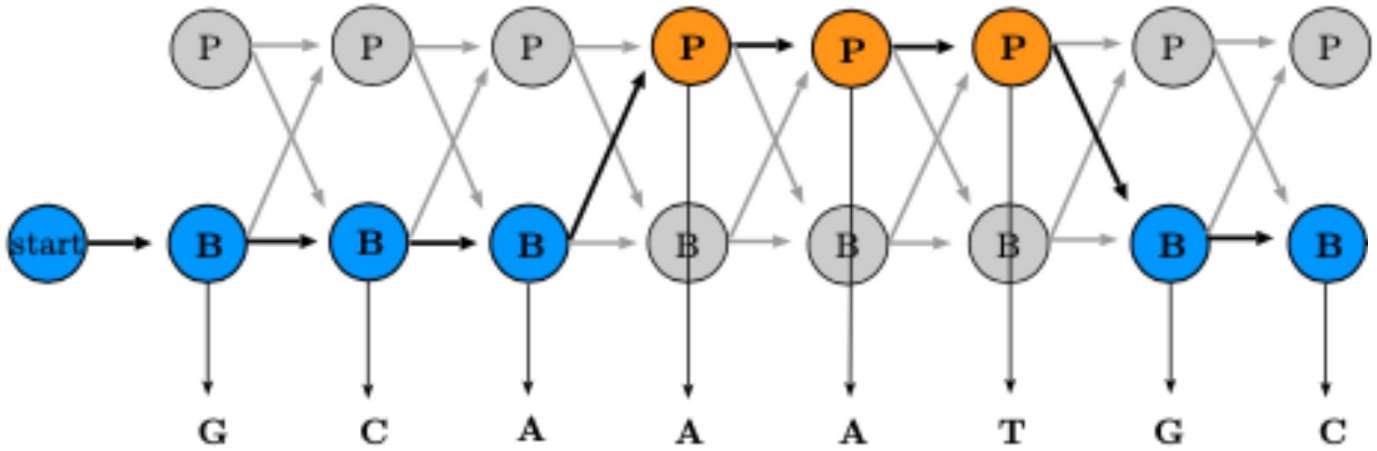
$E_k(b)$ = # of emissions of observation b from state k in the labeled training sequences

Maximum likelihood estimators:

$$\hat{a}_{kl} = \frac{A_{kl}}{\sum_{l'} A_{kl'}} \qquad \hat{e}_k(b) = \frac{E_k(b)}{\sum_{b'} E_k(b')}$$

Parameter estimation with hidden state labels

Example with one training sequence:



$$\hat{a}_{\mathrm{BP}} = \frac{1}{4} \qquad \hat{e}_{\mathrm{B}}(\mathrm{G}) = \frac{2}{5} \qquad \hat{e}_{\mathrm{B}}(\mathrm{T}) = 0$$

Parameter estimation with hidden state labels

In practice, some transitions/emissions may be unobserved in the training sequences and have 0 counts.

Therefore, add small pseudocounts to the above counts:

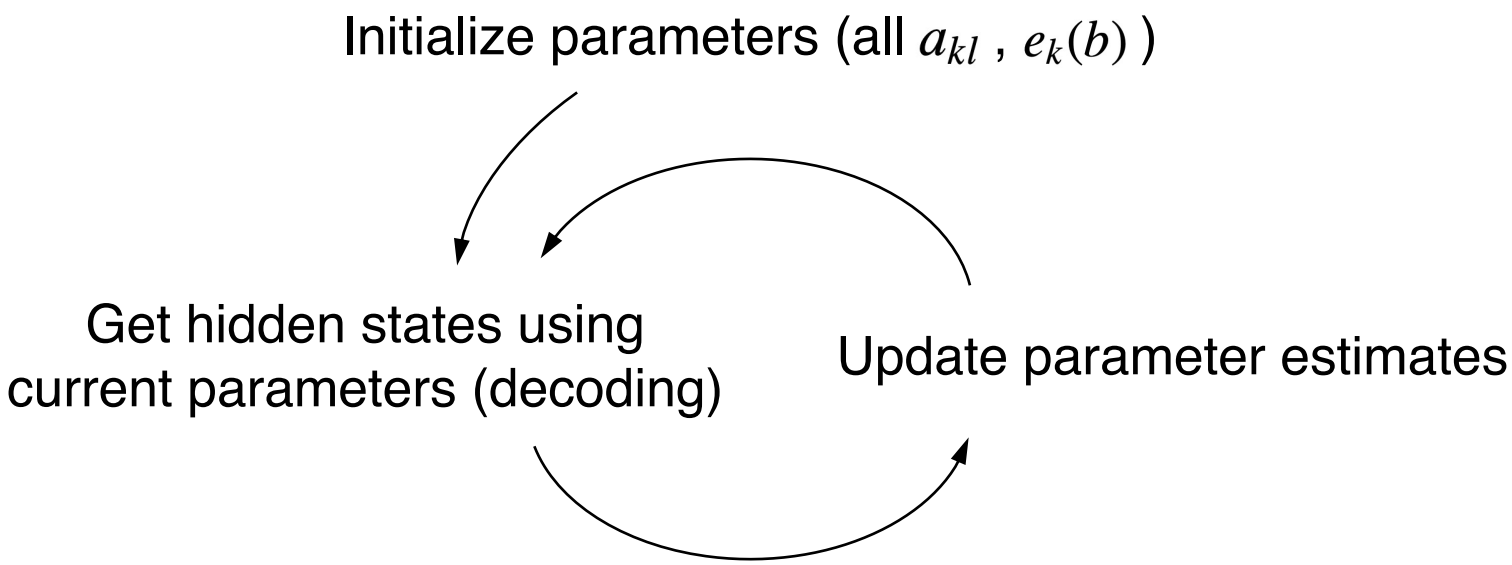
$$A_{kl} + r_{kl} \qquad\qquad E_k(b) + r_k(b)$$

Pseudocounts should reflect prior belief about the probabilities (small values for weak priors and large values for strong priors).

What if hidden states are unknown?

$$\hat{\theta} = \text{argmax}_{\theta} \prod_{j=1}^{n} P\left(x^{(j)} \middle| \theta\right)$$

Iterative process (converges to a local maximum):



What if hidden states are unknown?

Two approaches for updating parameters:

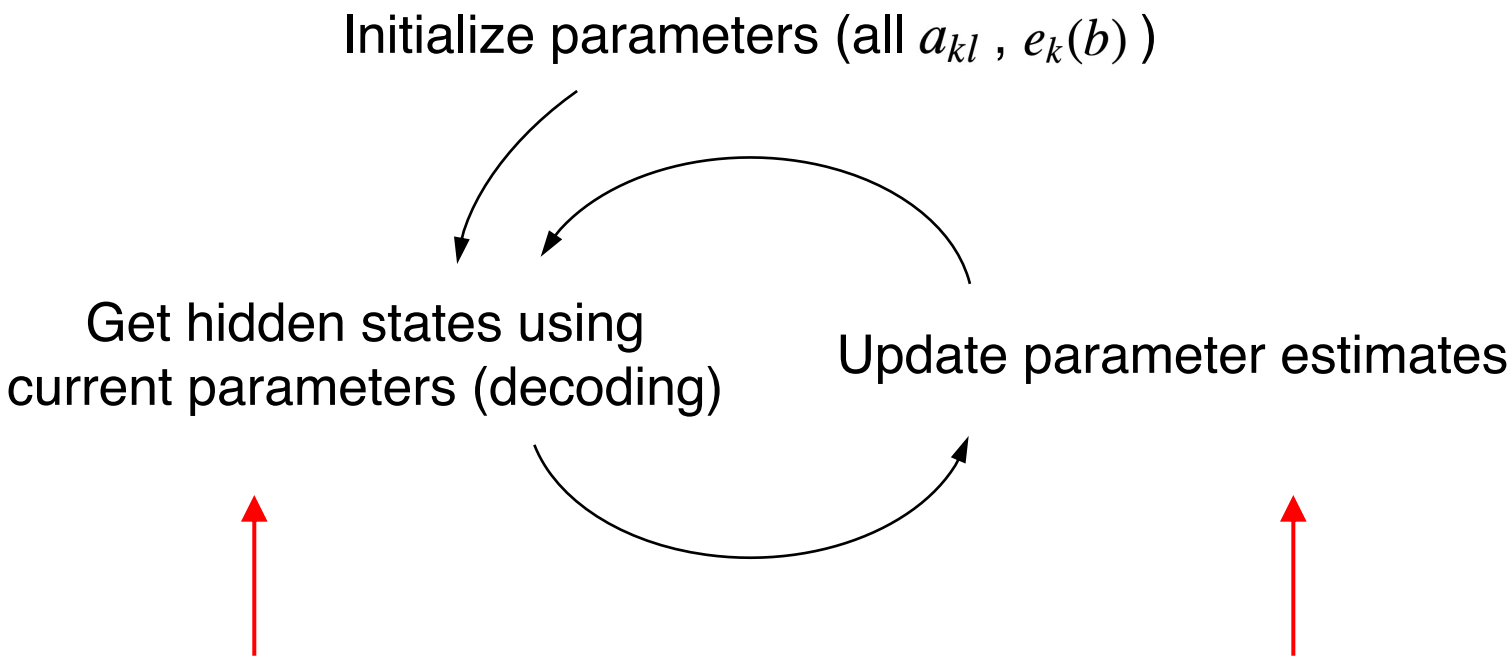
1. Viterbi training

   Considers only the most likely hidden state sequence at each iteration.

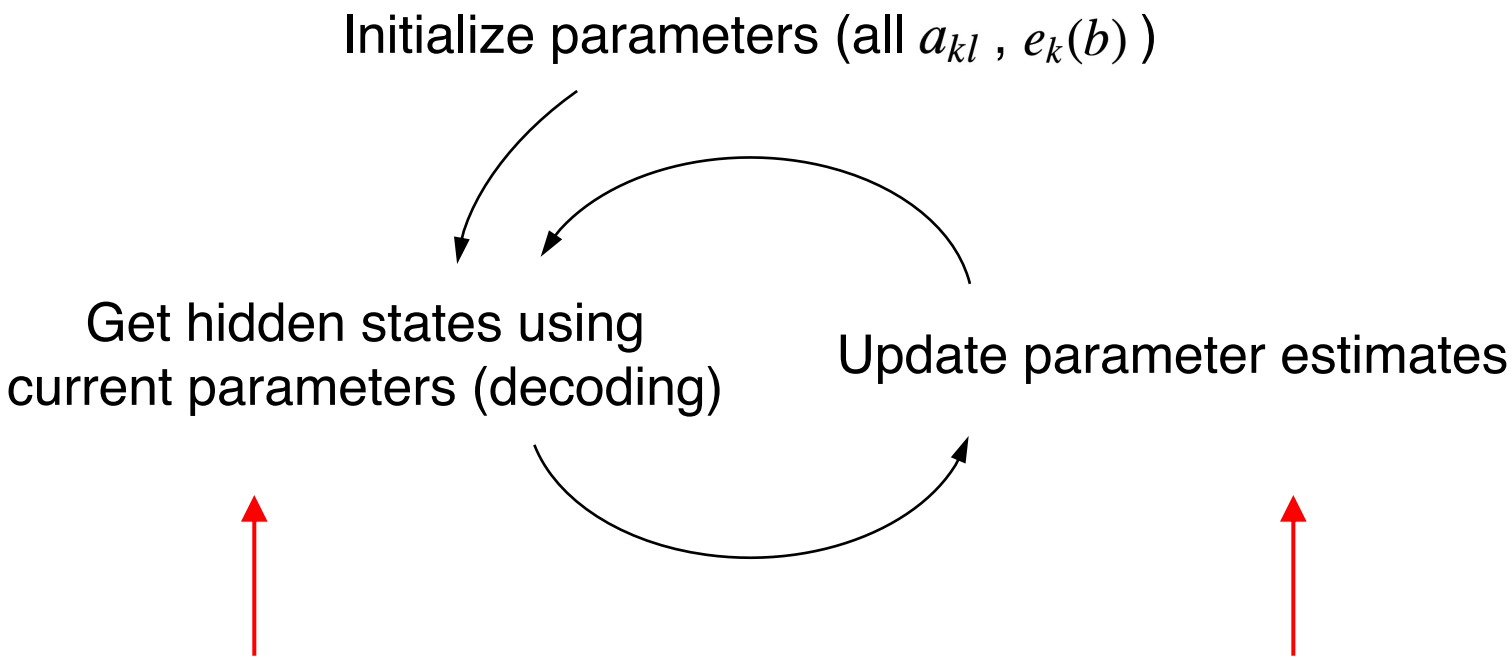
2. Baum-Welch algorithm

   Considers all possible hidden state sequences and their probabilities at each iteration (from forward-backward algorithm).

Viterbi training



Baum-Welch algorithm



Baum-Welch algorithm

Instead of counting transitions and emissions from most likely hidden state sequence, compute expected counts across all hidden state probabilities:

A_{kl} = EXPECTED # of transitions from state k to state l given current parameters

$E_k(b)$ = EXPECTED # of emissions of observation b from state k given current parameters

Then update parameter values as before:

$$\hat{a}_{kl} = \frac{A_{kl}}{\sum_{l'} A_{kl'}} \qquad \hat{e}_k(b) = \frac{E_k(b)}{\sum_{b'} E_k(b')}$$

Baum-Welch algorithm: emissions

Estimating $E_k(b)$: $E_k(b)$ = EXPECTED # of emissions of observation b from state k given current parameters

$$E_k(b) = \sum_{\{i|x_i=b\}} P(\pi_i = k|x) = \sum_{\{i|x_i=b\}} \frac{P(x, \pi_i = k)}{P(x)}$$

Remember:

$$\begin{aligned} P(x, \pi_i = k) &= P(x_1 \ldots x_i, \pi_i = k) P(x_{i+1} \ldots x_L | x_1 \ldots x_i, \pi_i = k) \\ &= \underbrace{P(x_1 \ldots x_i, \pi_i = k)}_{f_k(i)} \underbrace{P(x_{i+1} \ldots x_L | \pi_i = k)}_{b_k(i)} \end{aligned}$$

Baum-Welch algorithm: emissions

Estimating $E_k(b)$: $E_k(b)$ = EXPECTED # of emissions of observation b from state k given current parameters

$$E_k(b) = \sum_{\{i|x_i=b\}} P(\pi_i = k|x) = \sum_{\{i|x_i=b\}} \frac{P(x, \pi_i = k)}{P(x)}$$

$$= \frac{1}{P(x)} \sum_{\{i|x_i=b\}} f_k(i) b_k(i)$$

For multiple training sequences j:

$$E_k(b) = \sum_j \frac{1}{P(x^j)} \sum_{\{i|x_i^j=b\}} f_k^j(i) b_k^j(i)$$

Baum-Welch algorithm: transitions

Estimating A_{kl} : A_{kl} = EXPECTED # of transitions from state k to state l given current parameters

$$A_{kl} = \sum_i P(\pi_i = k, \pi_{i+1} = l | x) = \sum_i \frac{P(x, \pi_i = k, \pi_{i+1} = l)}{P(x)}$$

Split the numerator after position i :

$$P(x, \pi_i = k, \pi_{i+1} = l)$$

$$= \underbrace{P(x_1 \dots x_i, \pi_i = k)}_{f_k(i)} \underbrace{P(x_{i+1} \dots x_L, \pi_{i+1} = l | x_1 \dots x_i, \pi_i = k)}_{P(x_{i+1} \dots x_L, \pi_{i+1} = l | \pi_i = k)}$$

Baum-Welch algorithm: transitions

$$\underbrace{P(x_1 \dots x_i, \pi_i = k)}_{f_k(i)} \underbrace{P(x_{i+1} \dots x_L, \pi_{i+1} = l | x_1 \dots x_i, \pi_i = k)}_{P(x_{i+1} \dots x_L, \pi_{i+1} = l | \pi_i = k)}$$

$$= P(x_{i+1} \dots x_L | \pi_{i+1} = l) \underbrace{P(\pi_{i+1} = l | \pi_i = k)}_{a_{kl}}$$

Remember $b_k(i) = P(x_{i+1} \dots x_L | \pi_i = k)$, so split remaining term:

$$P(x_{i+1} \dots x_L | \pi_{i+1} = l) = \underbrace{P(x_{i+2} \dots x_L | \pi_{i+1} = l)}_{b_l(i+1)} \underbrace{P(x_{i+1} | \pi_{i+1} = l)}_{e_l(x_{i+1})}$$

Baum-Welch algorithm: transitions

$$A_{kl} = \sum_i \frac{P(x, \pi_i = k, \pi_{i+1} = l)}{P(x)}$$

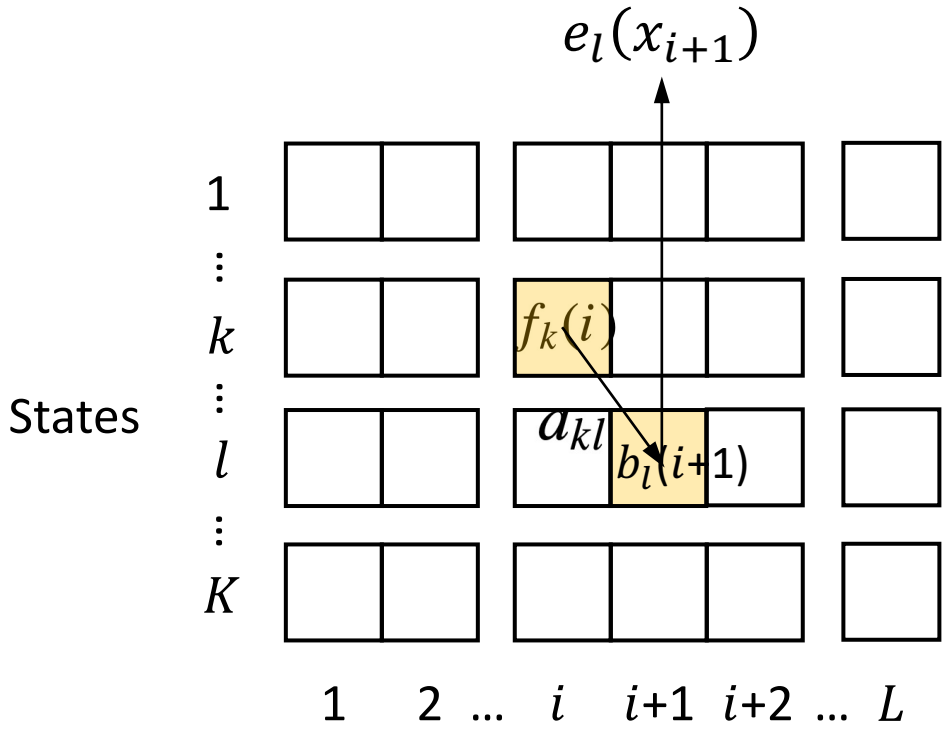
$$= \frac{1}{P(x)} \sum_i f_k(i) a_{kl} e_l(x_{i+1}) b_l(i+1)$$

For multiple training sequences j:

$$A_{kl} = \sum_j \frac{1}{P(x^j)} \sum_i f_k^j(i) a_{kl} e_l(x_{i+1}^j) b_l^j(i+1)$$

Baum-Welch algorithm: transitions

$$f_k(i)a_{kl}e_l(x_{i+1})b_l(i+1)$$



$$f_k(i) = P(x_1 \ldots x_i, \pi_i = k)$$

$$b_k(i) = P(x_{i+1} \ldots x_L | \pi_i = k)$$

Baum-Welch algorithm

Compute expected counts across all hidden state probabilities:

$$A_{kl} = \sum_j \frac{1}{P(x^j)} \sum_i f_k^j(i) a_{kl} e_l(x_{i+1}^j) b_l^j(i+1)$$

$$E_k(b) = \sum_j \frac{1}{P(x^j)} \sum_{\{i \mid x_i^j = b\}} f_k^j(i) b_k^j(i)$$

Then update parameter values as before:

$$\hat{a}_{kl} = \frac{A_{kl}}{\sum_{l'} A_{kl'}} \qquad \hat{e}_k(b) = \frac{E_k(b)}{\sum_{b'} E_k(b')}$$

Baum-Welch algorithm

Algorithm: Baum–Welch

Initialisation: Pick arbitrary model parameters.

Recurrence:

Set all the A and E variables to their pseudocount values r (or to zero).

For each sequence $j = 1 \ldots n$:

Calculate $f_k(i)$ for sequence j using the forward algorithm (p. 59).

Calculate $b_k(i)$ for sequence j using the backward algorithm (p. 60).

Add the contribution of sequence j to A (3.20) and E (3.21).

Calculate the new model parameters using (3.18).

Calculate the new log likelihood of the model.

Termination:

Stop if the change in log likelihood is less than some predefined threshold or the maximum number of iterations is exceeded.

Summary of algorithms

Find hidden states given observations and parameters:

- Viterbi algorithm $\pi^* = \underset{\pi}{\operatorname{argmax}} P(x, \pi)$
- Forward-backward algorithm (posterior decoding)

$$\hat{\pi}_i = \underset{k}{\operatorname{argmax}} P(\pi_i = k | x)$$

Find parameters given observations but unknown hidden states:

- Baum-Welch algorithm $\hat{\theta} = \operatorname{argmax}_\theta \prod_{j=1}^{n} P\left(x^{(j)} | \theta\right)$

Modeling choices

Topology: In practice, we may want to set certain state transitions to zero probability based on prior knowledge.

State duration: For a state with self-transition probability p, the distribution of state lengths/durations is geometric:

$$P(l \text{ residues}) = (1-p)p^{l-1}$$

The duration distributions can be adjusted in several ways, e.g. using generalized hidden Markov models (hidden semi-Markov models) and/or strings of bases rather than single bases as emissions.

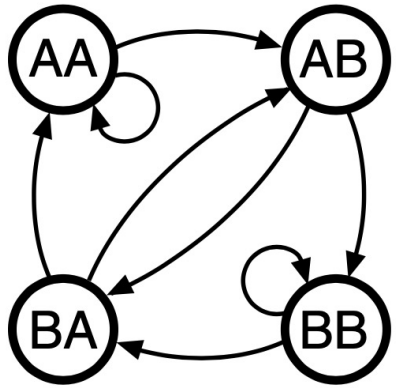
Modeling choices

Higher order Markov chains: Each event depends on the previous n events:

$$P(x_i|x_{i-1}, x_{i-2}, \ldots, x_1) = P(x_i|x_{i-1}, \ldots, x_{i-n})$$

Equivalent to a first-order Markov chain with n-tuples.

e.g. $n = 2$: ABBAB $\longrightarrow$ AB-BB-BA-AB



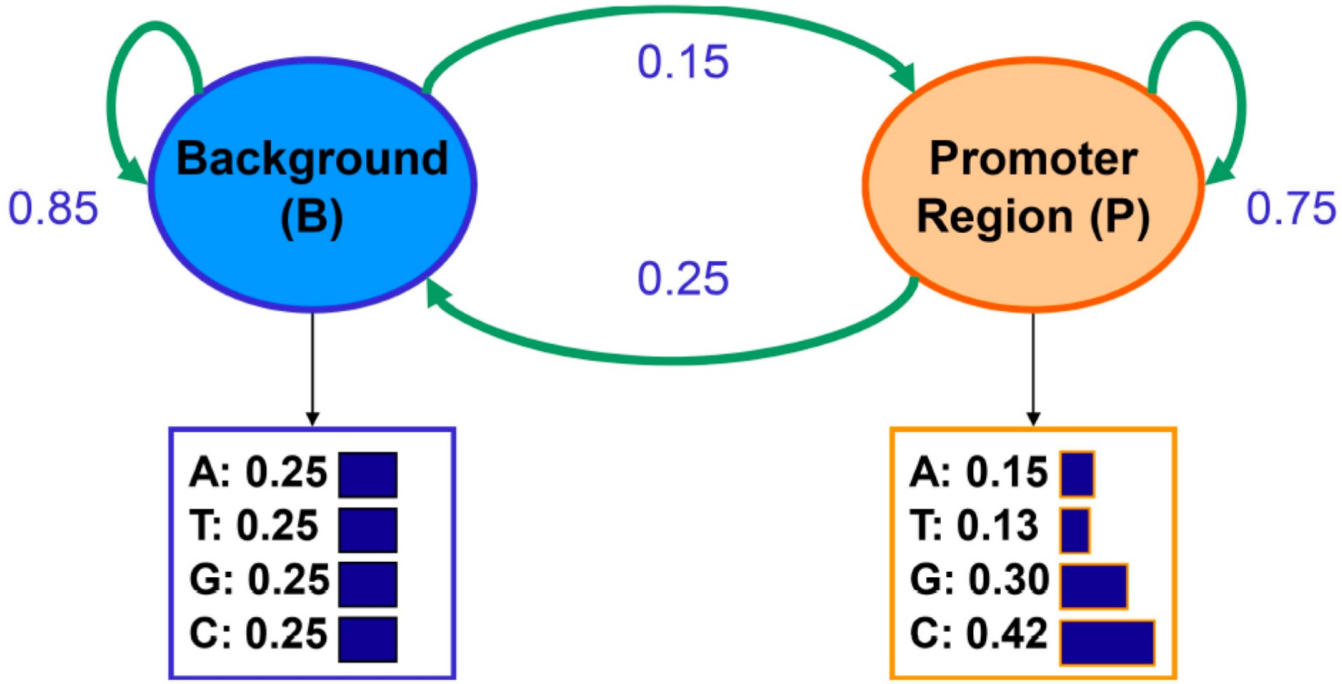
Applications of HMMs in genomics

Finding regions of interest within the genome (annotation), e.g.:

- Protein coding genes
  - Exons
  - Introns
  - 5’ and 3’ untranslated regions
- Regulatory elements
  - Promoters
  - CpG islands
  - Enhancers
- Evolutionarily conserved regions

Simple example: Finding promoters

Promoters (regions where gene expression is initiated) tend to have more C's and G's than other regions of the genome:



Example: Finding CpG islands

Two Markov models for comparing whole sequences (from last lecture):

CpG island model:

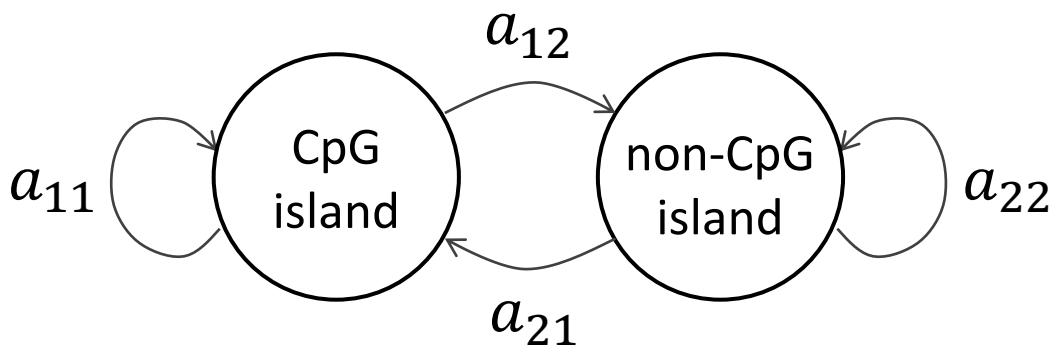
+	A	C	G	T
A	0.180	0.274	0.426	0.120
C	0.171	0.368	0.274	0.188
G	0.161	0.339	0.375	0.125
T	0.079	0.355	0.384	0.182

Non-CpG island model:

−	A	C	G	T
A	0.300	0.205	0.285	0.210
C	0.322	0.298	0.078	0.302
G	0.248	0.246	0.298	0.208
T	0.177	0.239	0.292	0.292

Example: Finding CpG islands

Two state HMM (from last lecture):



$e_1(\mathrm{A})$	$e_2(\mathrm{A})$
$e_1(\mathrm{C})$	$e_2(\mathrm{C})$
$e_1(\mathrm{G})$	$e_2(\mathrm{G})$
$e_1(\mathrm{T})$	$e_2(\mathrm{T})$

Example: Finding CpG islands

Adding ‘memory’ to the HMM:

Augment the state space with 4 CpG island hidden states (+) and 4 non-CpG island hidden states (-):

A+, C+, G+, T+, A-, C-, G-, T-

Option 1. Each hidden state encodes the **previous** base.

--> Emission probabilities reflect the chance of seeing each new base after that previous base.

Option 2. Each hidden state encodes the **current** base.

--> Transition probabilities reflect the chance of seeing each next base after the current base.

GENSCAN (Burge & Karlin 1997)

J. Mol. Biol. (1997) **268**, 78–94

JMB

Prediction of Complete Gene Structures in Human Genomic DNA

Chris Burge* and Samuel Karlin

Department of Mathematics Stanford University, Stanford CA, 94305, USA

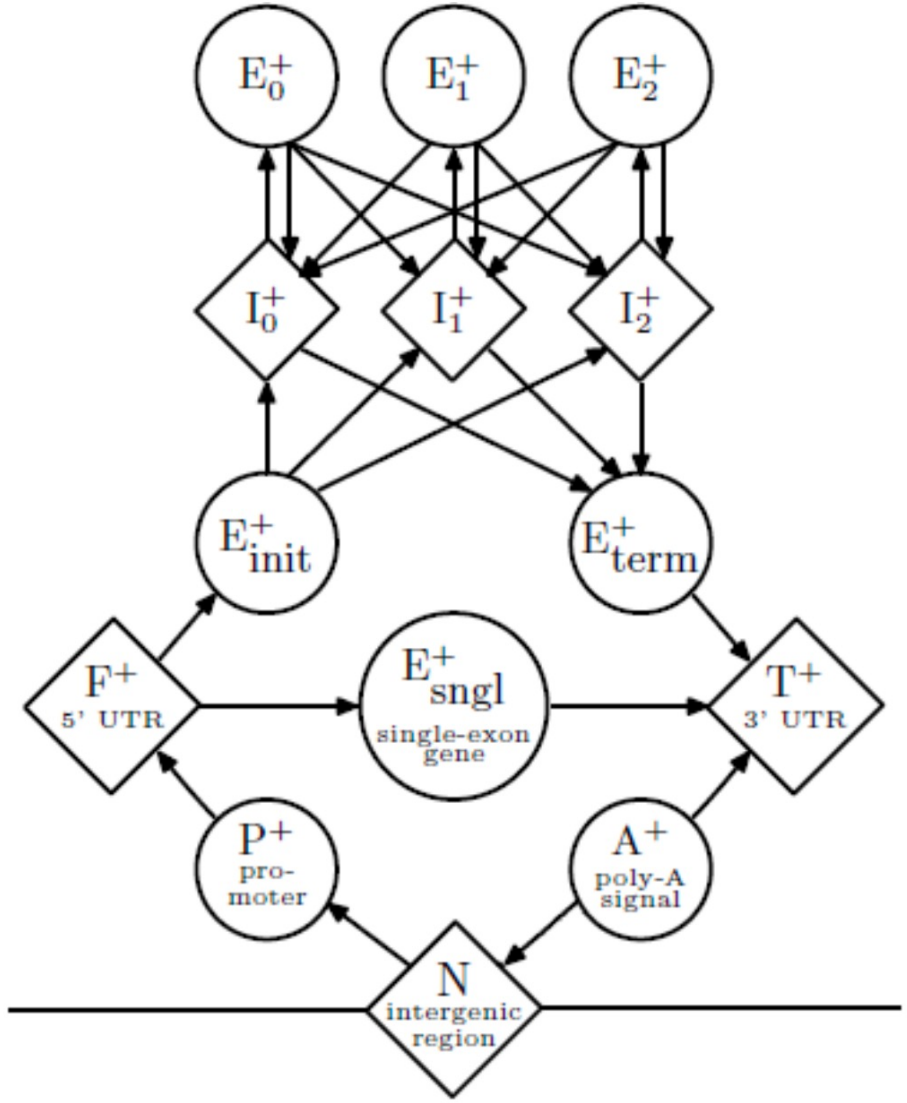
We introduce a general probabilistic model of the gene structure of human genomic sequences which incorporates descriptions of the basic transcriptional, translational and splicing signals, as well as length distributions and compositional features of exons, introns and intergenic regions. Distinct sets of model parameters are derived to account for the many substantial differences in gene density and structure observed in distinct C + G compositional regions of the human genome. In addition, new models of the donor and acceptor splice signals are described which capture potentially important dependencies between signal positions. The

https://www.sciencedirect.com/science/article/pii/S0022283697909517

GENSCAN (Burge & Karlin 1997)

HMM-based approach with modifications to better model transitions between exon and intron states:

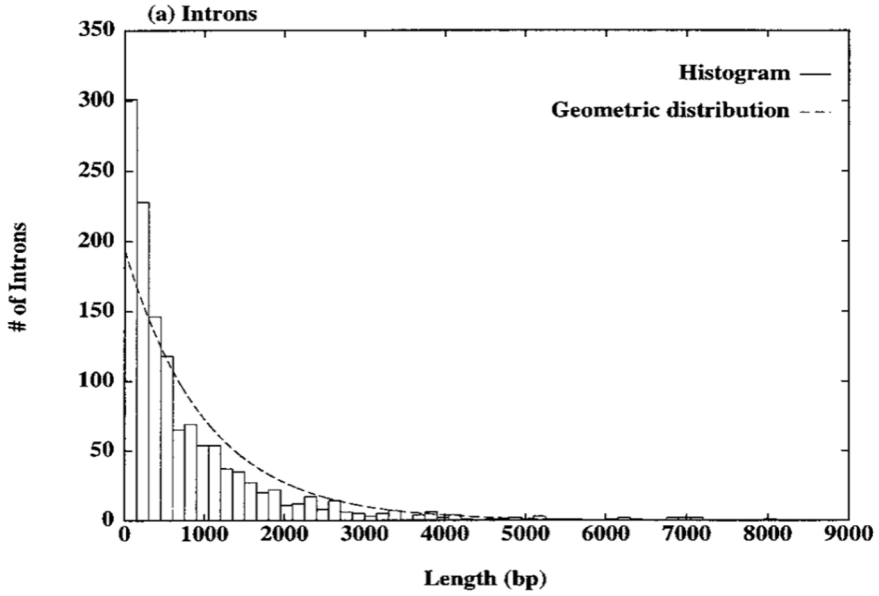
- Three different exon and intron states to account for reading frame; i.e. introns and exons that start at the first, second, or third base in a codon.
- Additional states for single-exon genes, 5' and 3' untranslated regions (UTRs), promoters, intergenic regions, etc.

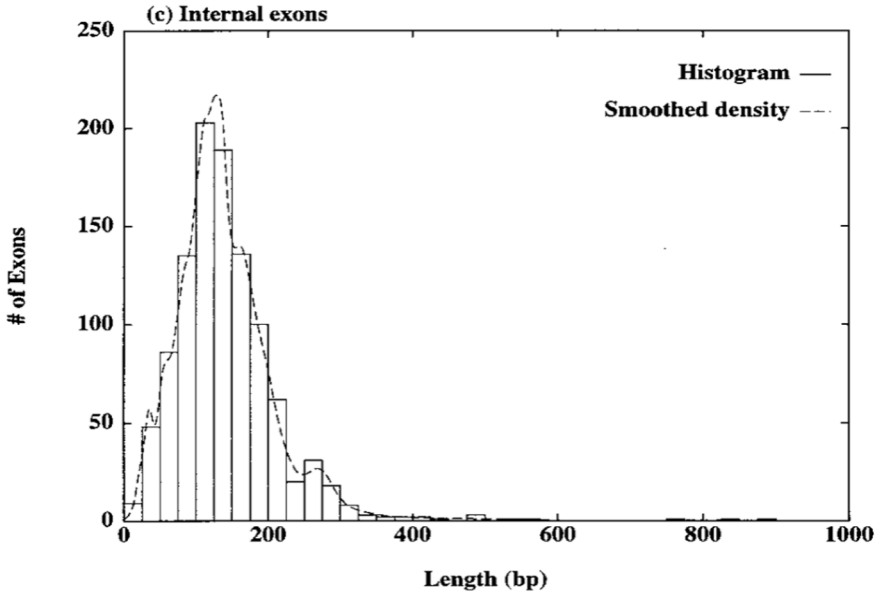


GENSCAN (Burge & Karlin 1997)

Generalized hidden Markov model (hidden semi-Markov model) to control the length distributions of exons:

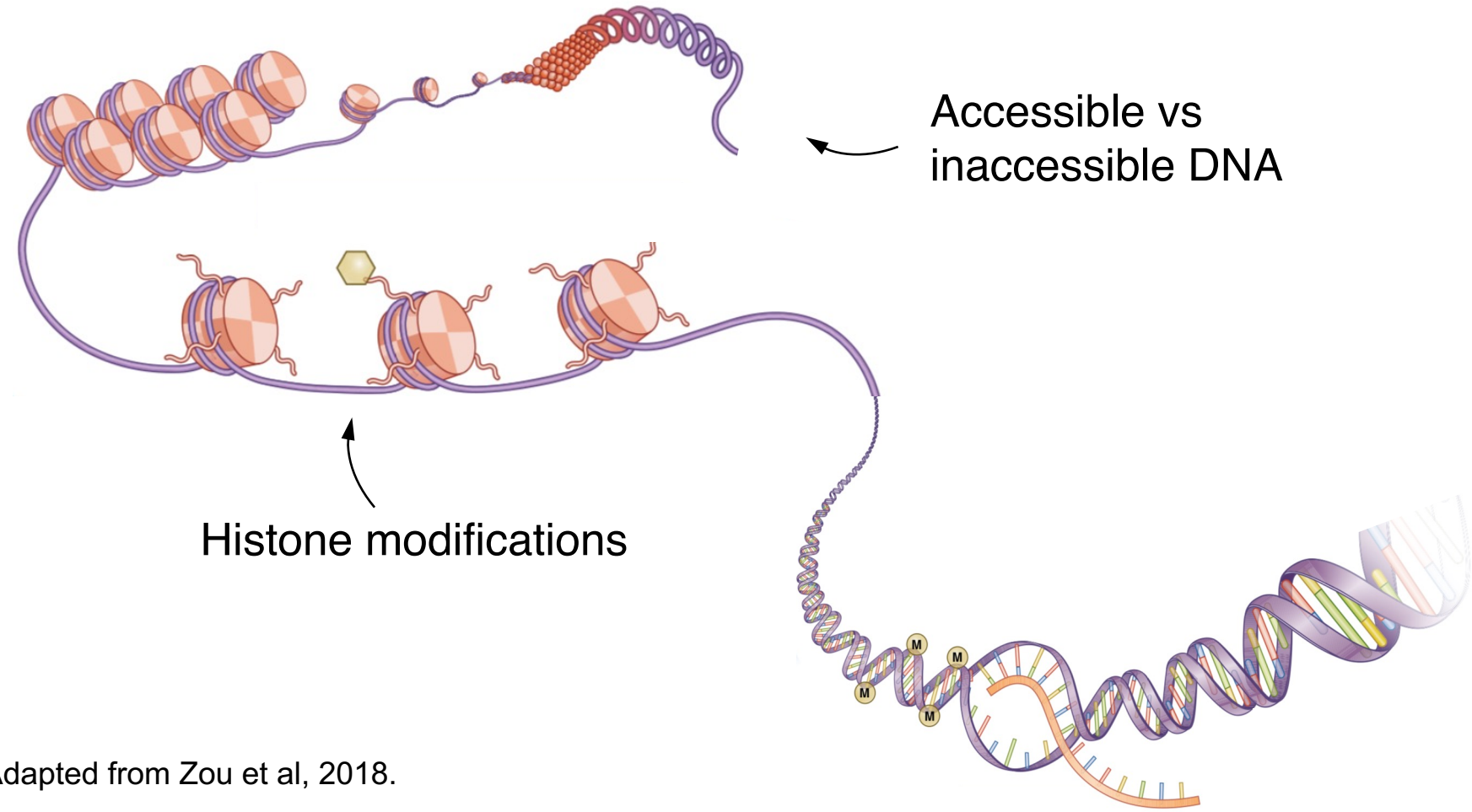
- When a new hidden state is reached, its length is randomly chosen from a length distribution corresponding to that type of state.
- After the number of emissions corresponding to the selected length, transitions into the next state follow the transition probabilities (with self-transition probabilities = 0).





ChromHMM (Ernst & Kellis 2010) approach

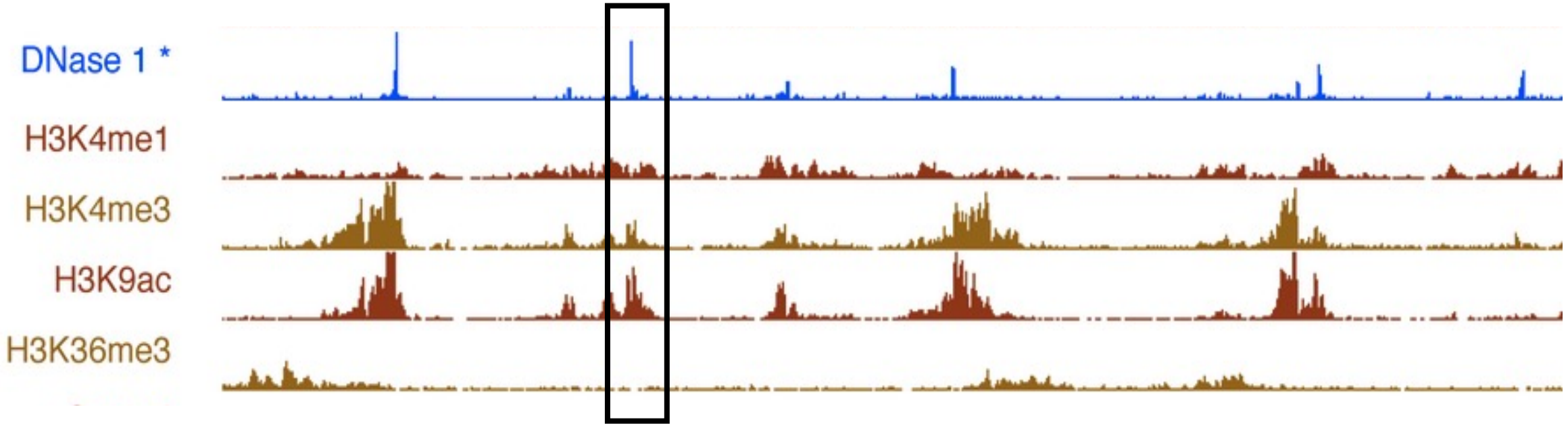
Find patterns of epigenetic modifications along the genome:



Adapted from Zou et al, 2018.

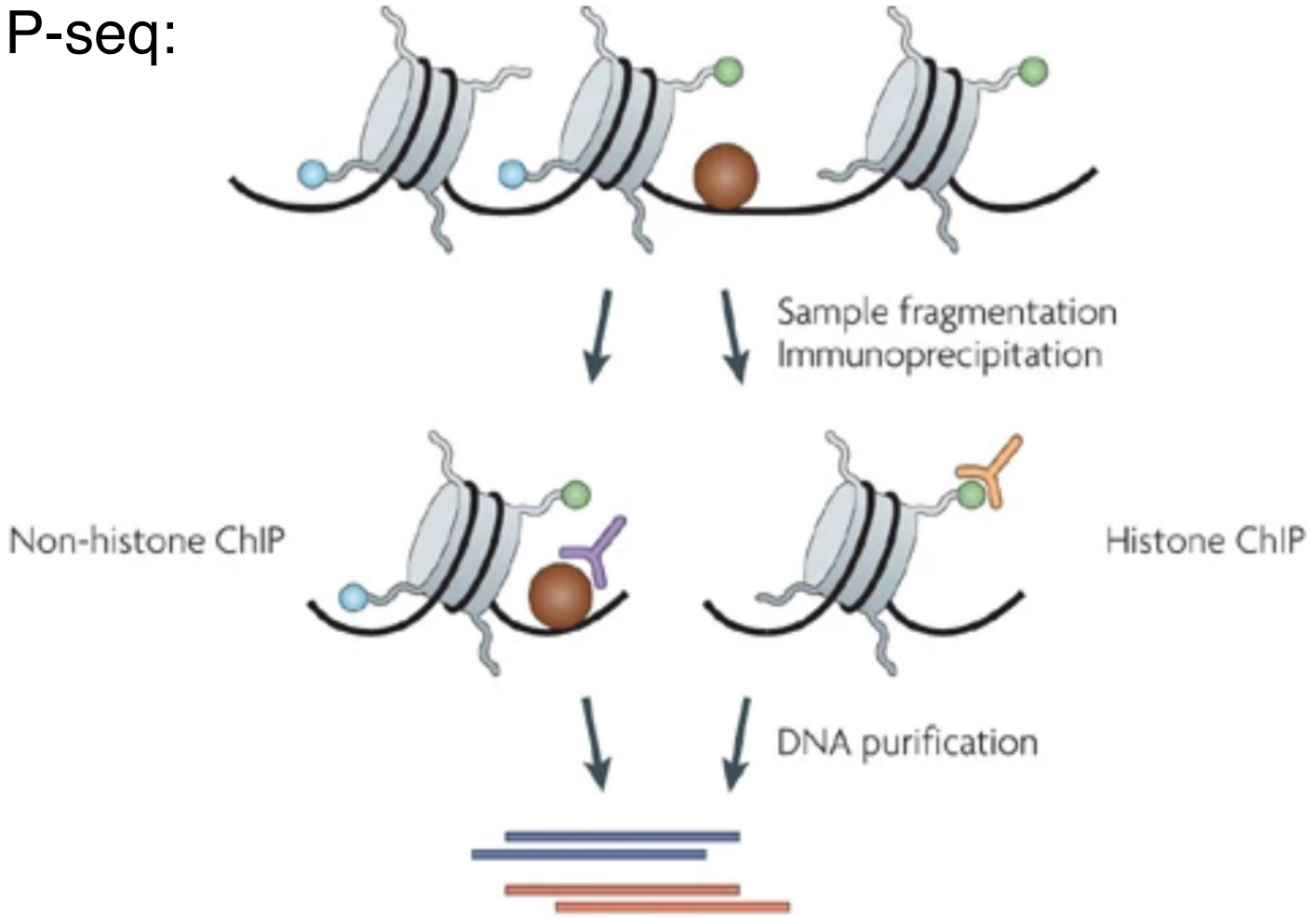
ChromHMM (Ernst & Kellis 2010) approach

Experimental measurements of epigenetic modifications:



Measuring histone modifications

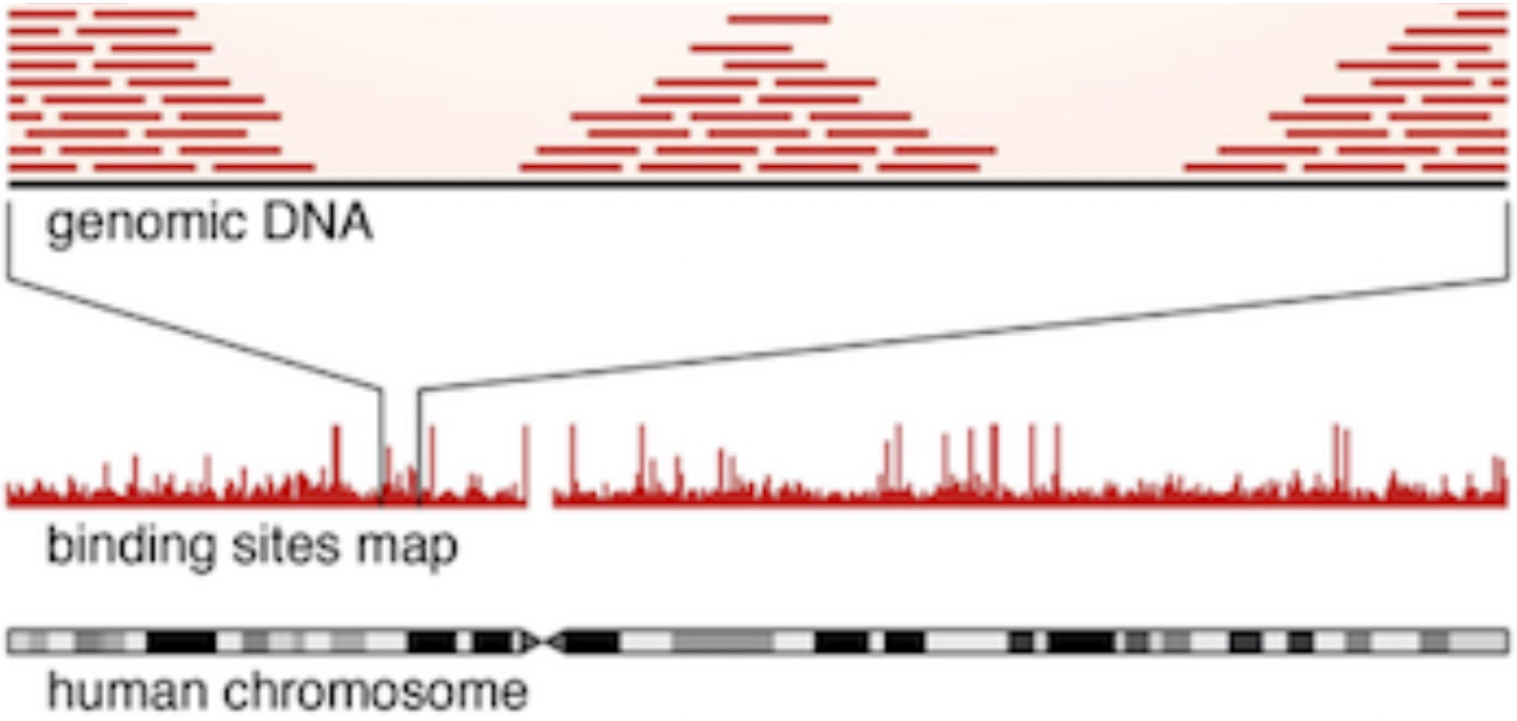
ChIP-seq:



Park, *Nature Reviews Genetics* 2009.

Measuring histone modifications

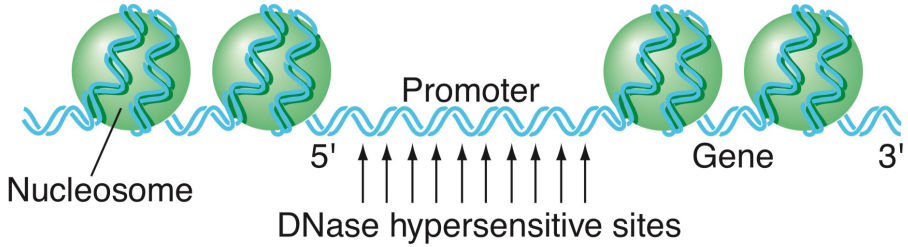
Sequence the bound fragments and map them back to the genome:



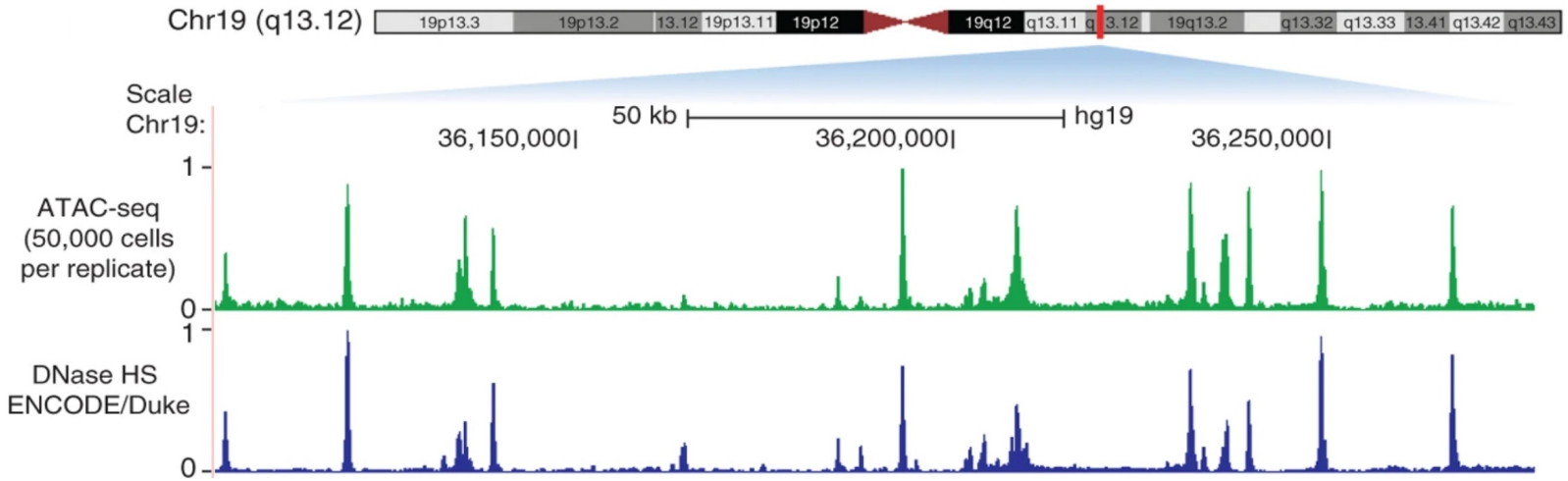
Harvard Chan Bioinformatics Core,
https://hbctraining.github.io/Intro-to-ChIPseq/lessons/01_Intro_chipseq_data_organization.html.

Measuring chromatin accessibility

Sequence short fragments of DNA from accessible regions (DNase-seq or ATAC-seq):



Example data:



Buenrostro, *Nature Methods* 2013.